

REST authentication



P.Mathieu

IUT de Lille

<http://www.iut-a.univ-lille.fr>

prenom.nom@univ-lille.fr

Authentication

BASIC authentication

Bearer authentication

Conclusion

Objectif

- ▶ REST : **RE**presentational **S**tate **T**ransfer
- ▶ On rappelle que
 - ▶ REST est un style architectural (pas un protocole)
 - ▶ Dans ce style on ne doit pas compter sur le serveur pour se souvenir des demandes précédentes (c'est dans l'acronyme ;-)).
- ▶ Chaque requête doit contenir toutes les informations nécessaires pour que le serveur puisse la comprendre.
- ▶ L'usage d'une session viole donc ce principe REST

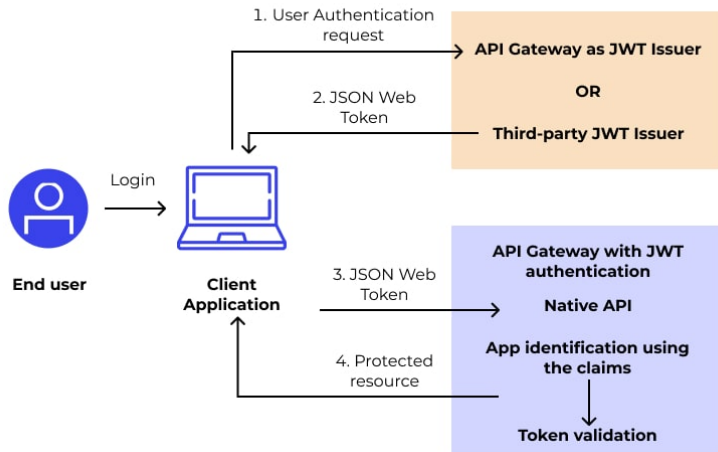
On dit que les applications REST sont “sans état” (Stateless)

Principe d'authentification par token

Pour éviter de conserver un état au niveau du serveur, on utilise un jeton d'authentification (token) qui sera fourni à chaque requête :

- ▶ On dissocie ainsi la phase d'authentification, de l'usage de l'application.
 - ❶ Un serveur d'authentification fournit un jeton en échange d'un login/mdp
 - ❷ Chaque requête HTTP envoie sa demande au serveur avec ce jeton, qui vérifie sa validité.
- ▶ REST est avant tout fait pour que les machines communiquent avec les machines !
- ▶ Cette technique est donc impérative dans les scénarii de serveur à serveur (applications interconnectées).
L'humain préférera "la page de login" classique.

JWT Authentication workflow



©<https://www.wallarm.com/what/>

Comment faire passer le token ?

De nombreuses techniques existent pour “passer” le token (param, header specif)
Néanmoins un header spécial HTTP est dédié à ces informations :

Authorization

Authorization: **<type>** **<token>**

Parmi les types possibles :

- ▶ Basic authentication : par login/mdp en clair
- ▶ Digest authentication : par login/mdp , hashage MD5
- ▶ Bearer authentication : par un token signé généré par un serveur d'authentification
- ▶ HOBA, Mutual, VAPID, SCRAM
- ▶

Chaque processus d'identification se déroule en deux phases

- ▶ Authentification : vérifier le token (par ex login/pwd ok)
- ▶ Autorisation : vérifier que l'utilisateur a le bon rôle pour accéder aux ressources

Il est important de bien distinguer les deux !

Deux codes d'erreur principaux sont liés au processus d'identification :

401 Unauthorized : token erroné ou absent

403 Forbidden : utilisateur connu, mais non autorisé

Quel que soit le token (BASIC, BEARER, etc ...) cela peut se mettre en place

- ▶ Via une configuration du conteneur (realms) :

Configuration à réaliser en 2 parties dans un serveur JEE :

- 1 décrire la méthode de stockage utilisés dans `context.xml`
(Memory, JDBC, LDAP, ...)
- 2 décrire les autorisations et la méthode de saisie dans `web.xml`
(quel rôle est associé à quelle page ?)

- ▶ Dans la partie applicative
soit à travers de tests à réaliser dans les servlets, soit via des filtres
(beaucoup de “boilerplate” code)

Authentication

BASIC authentication

Bearer authentication

Conclusion

Rappel sur BASE64

- ▶ But : transformer tout fichier binaire en texte pouvant transiter sur le réseau
- ▶ Base64 s'appuie sur 64 caractères : les lettres maj et minus ainsi que les chiffres, et les 2 caractères + /
- ▶ Chaque groupe de 24 bits est divisé en 4 groupes de 6 bits, correspondant à 4 caractères de l'alphabet possible ($2^6 = 64$).

- ▶ On complète avec des =

```
echo 'blanche neige et les 7 nains' | base64  
YmxhbmNoZSBuZWlnZSBldCBsZXMgNyBuYWlucwo=
```

- ▶ Base64URL : on remplace + par - et / par _
- ▶ Base64 ne protège en rien ! il est parfaitement réversible
- ▶ Unix possède une commande `base64` et Java une classe `Base64` pour encoder et décoder.

Principe du mode BASIC

- ▶ Le token est constitué du couple `login:mdp` encodé en base64URL.
- ▶ Il est envoyé par le client dans chaque requête HTTP
- ▶ Le token est rangé dans le header sous la forme

```
Authorization: Basic amVhbJpqZWFu
```

Curl (option `-u`)

```
curl http://localhost:8080/mem... -H 'Authorization: BASIC amVhbJpqZWFu'  
curl -u jean:jean http://localhost:8080/memoryAuthent/admin/page.jsp
```

Httpie (option `-a`)

```
http http://localhost:8080/mem... Authorization:'BASIC amVhbJpqZWFu'  
http -a jean:jean http://localhost:8080/memoryAuthent/admin/page.jsp
```

Avec le bon header (`WWW-Authenticate`), les navigateurs facilitent le travail ...

- ▶ Au premier appel sur une page protégée, l'utilisateur n'est pas authentifié
- ▶ le serveur renvoie alors une erreur 401 `Unauthorized` avec un header `WWW-Authenticate: Basic realm="entête popup de login"`
- ▶ le navigateur envoie alors une popup de saisie login/mdp , fabrique puis conserve le token
- ▶ Le navigateur se charge alors de l'envoyer automatiquement dans chaque requête (comme les cookies)

cool ... mais on rappelle que le navigateur ne sait faire que GET ou POST

Dans chaque méthode

```
String authorization = req.getHeader("Authorization");
if (authorization == null || !authorization.startsWith("Basic"))
    return false;
// on décode le token
String token = authorization.substring("Basic".length()).trim();
byte[] base64 = Base64.getDecoder().decode(token);
String decoded = new String(base64, StandardCharsets.UTF_8);
int idx = decoded.indexOf(':');
if (idx < 0) return false;
String login = decoded.substring(0, idx);
String pwd   = decoded.substring(idx + 1);

// et on vérifie ensuite qu'il correspond à quelqu'un qui a les droits
...
```

Inconvénient de BASIC

- ▶ Rien n'est protégé (base64 est réversible)
- ▶ Un pirate qui espionne les trames, récupère le login/mdp
- ▶ Pas de durée de validité de cette autorisation, ni de récupération possible

Le protocole HTTPS (avec SSL) est impératif quand BASIC est utilisé

Authentication

BASIC authentication

Bearer authentication

Conclusion

Principe

L'authentification se fait en deux temps :

- 1 On envoie dans un 1^{er} temps à un serveur d'authentification un couple login/password et on récupère un token signé en retour
- 2 Ce token est utilisé à la place du login/mdp pour toutes les autres requêtes

On ne vérifie plus le login/mdp mais la validité du token !

Le token est rangé dans le header sous la forme

```
Authorization: Bearer <token>
```

Bearer authentication est une “authentification au porteur”

Avantages

Trois avantages relativement à BASIC :

- ▶ Le token peut avoir une durée de vie limité.
s'il récupère le token, un pirate n'aura que cette durée (10mn) pour pirater
- ▶ On peut ajouter des infos variées dans le token
- ▶ On peut dissocier serveur d'authentification et serveur d'application
(se faire authentifier via Google ou Facebook par exemple)

L'important pour un Bearer token n'est pas d'être illisible mais d'être infalsifiable !!
⇒ token signé

Différentes formes d' Access token

Il existe de nombreuses formes d'Access tokens

- ▶ API tokens
- ▶ Simple Web Tokens (SWT, souvent UUID)
- ▶ Security Assertion Markup Language Tokens (SAML)
- ▶ OAuth tokens
- ▶ JSON Web Tokens (JWT)
- ▶ ...

JWT est un justificatif d'identité (token signé), standardisé (RFC 7519) , dont le token est constitué de 3 parties décrites en JSON.

Le token JWT

Un token JWT est constitué de 3 parties séparées par des points, chacune hachée en base64

```
header.payload.signature
```

- ▶ header (encodé en Base64) indique l'algo de hashage utilisé
- ▶ payload (encodé en Base64) fournit des données personnalisées (sub, exp, iat, iss, aud, prn, jti, typ)
- ▶ signature, constituée par hachage via l'algo choisi de la concaténation du header, du payload ET d'une clé secrète
- ▶ La signature la plus fréquente est HMAC + SHA256 appelée aussi HS256 (mais plein d'autres sont possibles.)

Bearer authentication

Token JWT

 JWT

Debugger Libraries Introduction Ask Get a T-shirt!

Crafted by  Auth0

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJJSb2x1IjoiaWRTaW4iLCJjc3N1ZXIiOiJMUERBMkklLCJzdWIiOiJyNywiZXhwIjoxNjE1MDQ4ODk3LCJpYXQiOiJlMTUwNDg4OTd9.yxLUZ58EPiGfEwzCNgtpcI3HRyiM0rjJ-TKuOPxT01g
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD: DATA

```
{  "Role": "Admin",  "Issuer": "LPDA2I",  "sub": 227,  "exp": 1615048897,  "iat": 1615048897}
```

VERIFY SIGNATURE

```
HMACSHA256(  base64UrlEncode(header) + "." +  base64UrlEncode(payload),  blanche-neige)
```

☐ secret ☐ base64 encoded

Signature Verified

SHARE JWT

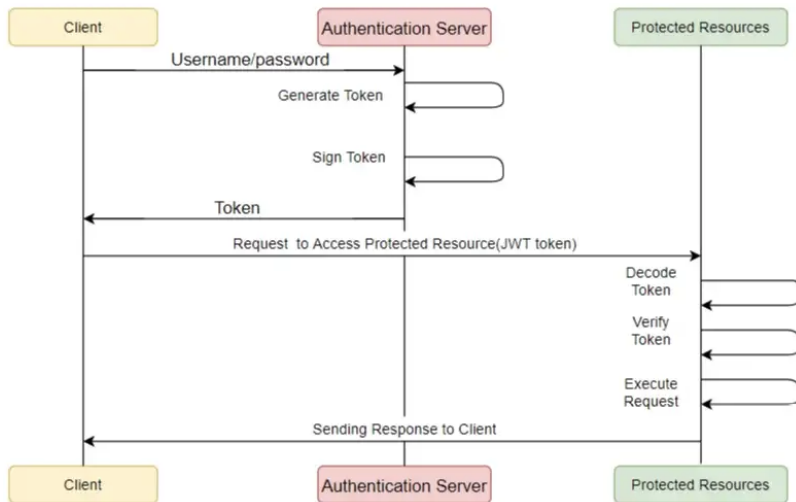
P.Mathieu (IUT Info, ULille)

REST authentication

20 / 25

Bearer authentication

Authentification avec un token signé



©<https://medium.com/@akshaey005>

De nombreux claims complémentaires peuvent être utilisés dans le header

- ▶ L'encodage/décodage d'un token JWT nécessite du “boilerplate code” assez technique.
- ▶ Il est préférable d'utiliser une librairie dédiée
- ▶ Il en existe plusieurs, selon les langages et technologies souhaitées.

Pour Java/JEE :

- ▶ jjwt <https://github.com/jwtkt/jjwt>
- ▶ java-jwt <https://github.com/auth0/java-jwt>
- ▶ io.jsonwebtoken <https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt>

Authentication

BASIC authentication

Bearer authentication

Conclusion

Conclusion

Pour être "stateless" il faut un mécanisme de "jeton" passé à chaque requête
Trois grandes familles co-existent :

- ▶ les **APIKey**. Sans durée de vie, en général du base64 sur UUID. Une fois généré, il n'y a rien à en tirer. Il faut juste sa présence dans la base. Il faut donc une colonne token dans la table des utilisateurs en plus du login/pwd. En général on les passe en paramètre de la requête HTTP (ex : NASA)
- ▶ les **"Basic authent"** avec `base64(login:pwd)`. Ils n'ont pas non plus de date de validité. Ils sont décodés à chaque requête afin de vérifier la correction du login/pwd. Pas de colonne supplémentaire à la table users. En général on les passe dans le header `Authorization: Basic <token>`
- ▶ les **tokens avec signature**. L'archétype étant **JWT** qui contient non seulement une signature mais une durée de validité A chaque requête on vérifie les deux ! En général on les passe dans le header `Authorization: Bearer <token>`

Conclusion

Les mécanismes d'authentification deviennent de plus en plus complexes (authentification multifactorielle). D'où la création de `single sign-on providers` (SSO) qui s'occupent de tout cela.

Avantages :

- ▶ On n'a plus à gérer ses mots de passe et donc moins de risques de violation de données
- ▶ On n'a plus à écrire les procédures complexes de login et logout
- ▶ Les utilisateurs n'ont pas besoin d'un nouveau compte

Authentifier est devenu un métier ;-)